

PROJECT REPORT

THIRANEX SECURE LOGIN SYSTEM

1. Abstract

The Thiranex Secure Login System is a web-based authentication project developed using Python Flask, HTML, CSS, and SQLite. The system allows new users to register an account and existing users to log in using their registered credentials. After successful authentication, the user is redirected to a protected dashboard. The project demonstrates the basic concepts of web authentication, session management, database connectivity, form handling, and user interface design.

2. Introduction

User authentication is an important part of many web applications. A login system helps control access to pages and services that should only be available to registered users. This project implements a simple authentication workflow using Flask as the backend framework and SQLite as the database.

3. Problem Statement

Many web applications need a simple method to register users, verify their login details, maintain a login session, and restrict access to protected pages. The purpose of this project is to develop a simple authentication system that demonstrates these functions in an organized and easy-to-understand way.

4. Objectives

- To create a user registration page.
- To create a user login page.
- To store registered user information in a database.
- To validate login details.
- To provide a protected dashboard for authenticated users.
- To implement logout and session handling.
- To create a simple and user-friendly interface.
- To understand Flask, templates, CSS, and SQLite integration.

5. Technologies Used

Technology	Purpose	Use in Project
Python	Programming language	Backend application logic

Flask	Web framework	Routes, requests, sessions and application control
HTML5	Web structure	Login, registration and dashboard pages
CSS3	Web styling	Layout, colors, cards and forms
SQLite	Database	Stores registered user information
Jinja2	Template engine	Displays dynamic content in HTML
python-dotenv	Environment configuration	Loads configuration values from .env

6. System Requirements

6.1 Hardware Requirements

Computer/Laptop

Minimum 4 GB RAM

At least 500 MB free storage

Keyboard and mouse

Internet connection for package installation

6.2 Software Requirements

Windows/Linux/macOS

Python 3.x

Visual Studio Code

Microsoft Edge or another modern browser

Flask and required Python packages

7. Project Structure

The project is organized into backend, template, styling, database, and configuration files.

.env/ – Python virtual environment.

static/style.css – CSS file for the user interface.

templates/base.html – Common HTML layout.

templates/login.html – Login page.

templates/register.html – Registration page.

templates/dashboard.html – Protected dashboard page.

app.py – Main Flask application.

.env – Environment configuration.

.gitignore – Files that should not be uploaded to Git.

requirements.txt – Required Python packages.

users.db – SQLite database.

8. System Workflow

1. User opens the application.
2. New user selects Register and enters the required details.
3. The application validates the registration data.
4. User information is stored in the SQLite database.
5. The user opens the Login page and enters the registered credentials.
6. The application checks the credentials.
7. For valid credentials, a session is created and the user is redirected to the dashboard.
8. The user can log out, which ends the active session.

9. Modules

9.1 Registration Module

This module allows a new user to create an account. The submitted information is validated before it is stored in the database.

9.2 Login Module

This module accepts user credentials and checks them against the registered account information. Successful authentication creates a user session.

9.3 Dashboard Module

The dashboard is the protected page shown after successful login. It demonstrates access control for authenticated users.

9.4 Logout Module

The logout function terminates the current login session and prevents continued access to protected content.

9.5 Database Module

SQLite is used to maintain user account information. The database file in the project is users.db.

10. Implementation

The application is implemented with Flask routes and HTML templates. The templates are stored inside the templates folder, while the CSS file is stored inside the static folder. Flask connects the web pages with the backend logic and database. Sessions are used to remember authenticated users while they use the application.

11. Security Considerations

Use a secret key through environment configuration.

Keep sensitive configuration values out of public source code.

Add .env to .gitignore.

Validate registration and login inputs.

Protect dashboard routes using authentication checks.

Use password hashing in the final application instead of storing plain-text passwords.

Do not use real personal passwords in screenshots or demonstration data.

12. Testing

Test Case	Input/Action	Expected Result	Status
Registration	Enter valid new-user details	Account is created	Pass/Verify
Login	Enter valid credentials	Dashboard opens	Pass/Verify
Invalid Login	Enter incorrect credentials	Access is rejected	Pass/Verify
Dashboard Protection	Open dashboard without login	User is redirected/restricted	Pass/Verify
Logout	Click Logout	Session ends	Pass/Verify

13. Advantages

Simple and easy to understand.

Uses a lightweight Flask framework.

Uses SQLite, so no separate database server is required.

Separates HTML templates and CSS.

Demonstrates basic authentication and session management.

Can be extended with additional security features.

14. Limitations

It is a basic authentication project and is not intended to be a complete production identity-management system.

Additional security controls are required for production deployment.

Email verification and password reset are not included in the basic version.

The exact security level depends on the authentication and password-storage implementation in app.py.

15. Future Enhancements

Email verification.

Forgot-password and password-reset functionality.

Two-factor authentication.

Login attempt limiting and account lockout.

Admin and user roles.

User profile management.

Improved database security.

Cloud deployment.

16. Conclusion

The Thiranex Secure Login System is a practical web development project that demonstrates how a Flask application can provide registration, login, session management, logout, database connectivity, and a protected dashboard. The project provides a foundation for learning web authentication and can be extended with stronger security and user-management features.